SWCON221 Microservice Programming

Lecture 8

Build and Push Images

2026. 9. 1

Sungwon Lee

Department of Software Convergence

경희대학교
KYUNG HEE UNIVERSITY

Contents

- Nginx Web-server
- Python Web-server
- JavaScript Web-server
- C++ Programmers
- Push Image {to Docker Hub}

Reading Articles

- Get Started, Part 3: Share the applications

  https://docs.docker.com/get-started/workshop/04_sharing_app/#docker-composeym

- Pushing and Pulling to and from Docker Hub

  https://ropenscilabs.github.io/r-docker-tutorial/04-Dockerhub.html

Dockerfile

```
FROM ubuntu:latest
MAINTAINER Dr.Sungwon "drsungwon@khu.ac.kr"
RUN apt-get update
RUN apt-get install -y nginx
RUN echo "this is a ubuntu container"
WORKDIR /etc/nginx
CMD ["nginx", "-g", "daemon off;"]
EXPOSE 80
```

Command Hints

```
docker build --tag myubuntu:1.0 .
docker image ls
docker run --name myubuntu-nginx -d -p 4000:80 myubuntu:1.0
docker container ls
docker stats
```

```
docker container stop
docker container rm
docker image rm
#execute web-browser & access to http://localhost:4000
```

🌟HOMEWORK🌟

Dockerfile

```
FROM python:latest
MAINTAINER Dr.Sungwon "drsungwon@khu.ac.kr"
COPY . /app
RUN apt-get update
RUN echo "this is a python web server container"
CMD ["python", “/app/webserver.py"]
EXPOSE 9000
```

HOMEWORK

webserver.py

```
import socket

HOST, PORT = '', 9000

listen_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
listen_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
listen_socket.bind((HOST, PORT))
listen_socket.listen(1)
print ('Serving HTTP on port', PORT, '...')
while True:
    client_connection, client_address = listen_socket.accept()
    request = str(client_connection.recv(1024),'utf-8')
    print (request)

    http_response = """₩
HTTP/1.1 200 OK

Hello world!
"""
    client_connection.sendall(bytes(http_response,'utf-8'))
    client_connection.close()
```

🌟HOMEWORK🌟

Command Hints

docker build --tag mywebserver:1.0 .

docker image ls

docker run --name mypyserver -d -p 8090:9000 mywebserver:1.0

docker container ls

docker stats

docker container stop

docker container rm

docker image rm

#execute web-browser & access to http://localhost:8090

Dockerfile

```
# https://nodejs.org/ko/docs/guides/nodejs-docker-webapp/
FROM node:10
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 8080
CMD [ "node", "server.js" ]
```

server.js

```
# https://nodejs.org/ko/docs/guides/nodejs-docker-webapp/
'use strict';
const express = require('express');
const PORT = 8080;
const HOST = '0.0.0.0';
const app = express();
app.get('/', (req, res) => {
  res.send('Hello world₩n');
});
app.listen(PORT, HOST);
console.log(`Running on http://${HOST}:${PORT}`);
```

package.json

```
# https://nodejs.org/ko/docs/guides/nodejs-docker-webapp/
{
  "name": "docker_web_app",
  "version": "1.0.0",
  "description": "Node.js on Docker",
  "author": "First Last <first.last@example.com>",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.16.1"
  }
}
```

.dockerignore

```
# https://nodejs.org/ko/docs/guides/nodejs-docker-webapp/
node_modules
npm-debug.log
```

Command Hints

```
docker build --tag myjsserver:1.0 .
docker image ls
docker run --name myjswebserver -d -p 8080:8080 myjsserver:1.0
docker container ls
docker stats
docker container stop
docker container rm
docker image rm
#execute web-browser & access to http://localhost:8080
```

gcc / boost.asio

- gcc docker image
  https://hub.docker.com/_/gcc

- boost.asio network and low-level I/O C++ APIs
  https://www.boost.org/

- boost.asio example
  https://www.c-sharpcorner.com/article/socket-programming-in-cpp-using-boost-asio-tcp-server-and-client/

CLI Interface

Pushing and Pulling to and from Docker Hub

Lesson Objectives

- Understanding where images come from
- Pulling a Docker image from Docker Hub
- Pushing a Docker image to Docker Hub

Getting an image from Docker Hub

Docker Hub is the place where open Docker images are stored. When we ran our first image by typing

```
docker run --rm -p 8787:8787 rocker/verse
```

the software first checked if this image is available on your computer and since it wasn't it downloaded the image from Docker Hub. So getting an image from Docker Hub works sort of automatically. If you just want to pull the image but not run it, you can also do

```
docker pull rocker/verse
```

🌟HOMEWORK🌟

GUI Interface

Step.1 Log-in to Docker Hub using Docker Desktop dashboard

Step.2 Create a Docker image for ‘myapp of Lecture.7’

docker build -t drsungwon/myapp:1.0 .

```
drsungwon~$ docker build -t drsungwon/myapp:1.0 .

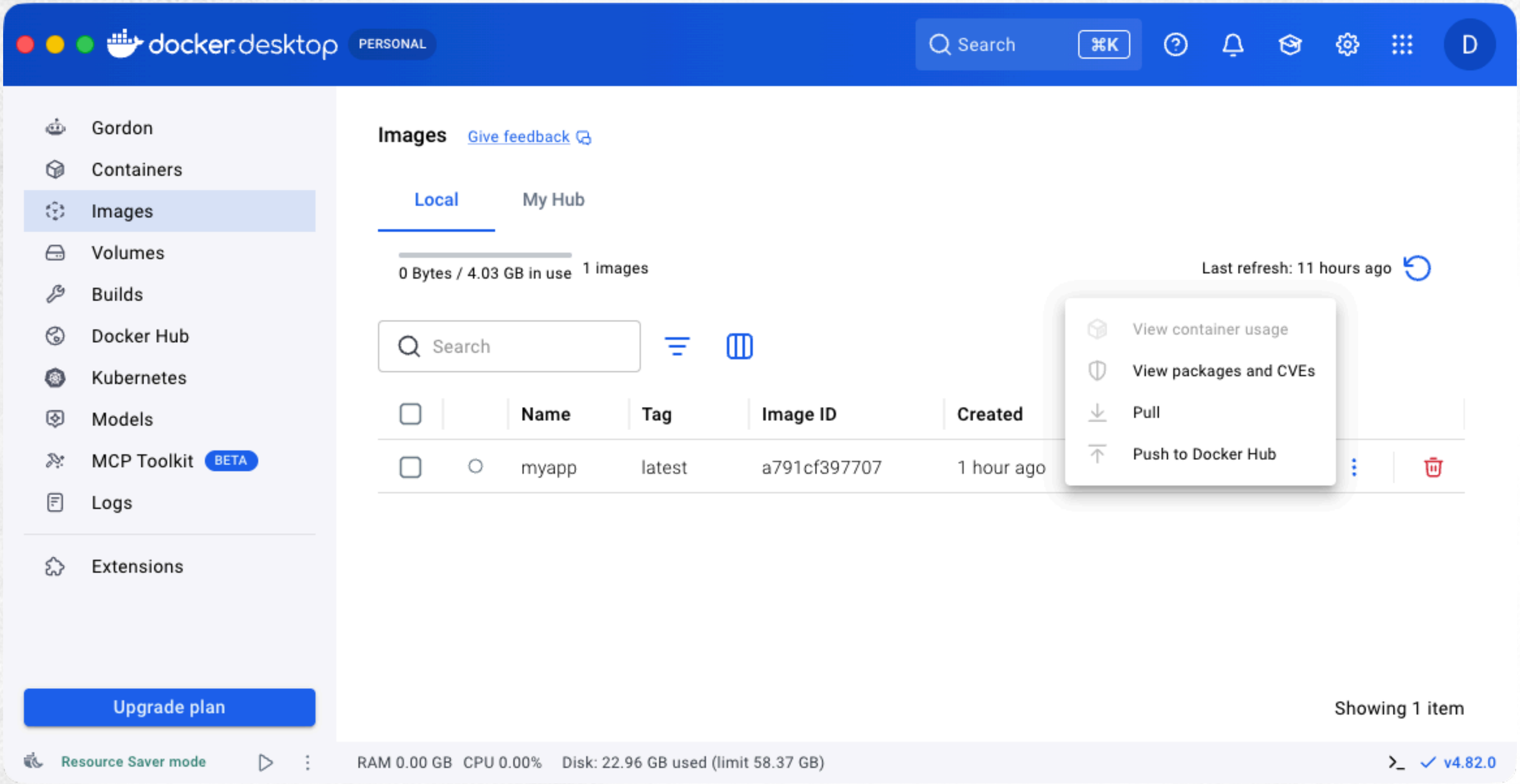
[+] Building 1.4s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 95B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/python:3
 => [internal] load build context
 => => transferring context: 176B
 => [1/2] FROM docker.io/library/python:3@sha256:ce21f64c4c3ae5743ddd5f4d4d9ca5614fddcc4f8c6e32ff2a7ff9a2e8744e8d
 => CACHED [2/2] COPY . /app
 => exporting to image
 => => exporting layers
 => => writing image sha256:80db1c13129942d707a6a09d47a6b5f2ff158048454f221fe09f064a5715b2ab
 => => naming to docker.io/drsungwon/myapp:1.0

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
drsungwon~$
```

HOMEWORK

GUI Interface

Step.3 Push using Docker Desktop dashboard



Thank you

경희대학교
KYUNG HEE UNIVERSITY